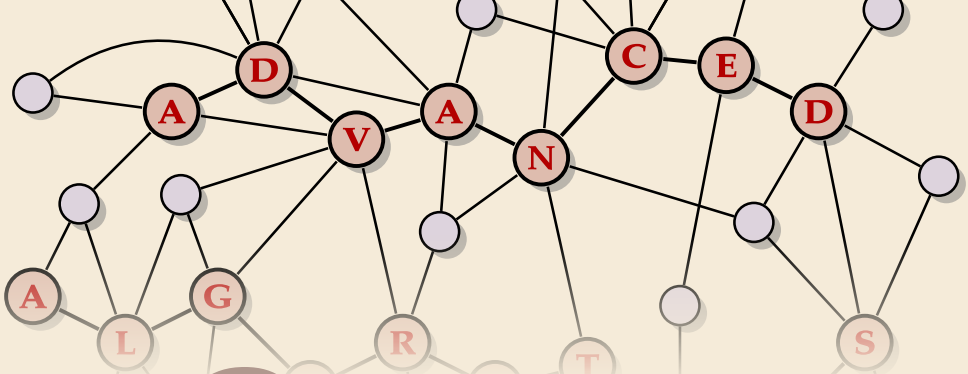


3 Pseudopolynomial Algorithms

Advanced Algorithms · Winter 2026–27

Prof. Dr. Sebastian Wild

CS627 (Winter 2026–27)
Philipps-Universität Marburg
version `2026-10-22 14:16`

Outline

3 Pseudopolynomial Algorithms

3.1 Integer Problems

3.2 Knapsack

3.3 Strong NP-hardness

3.1 Integer Problems

Integer Problems

Definition 3.1 (Integer-Input Problem)

A problem $L \subseteq U$ for which (part of the) input is a *sequence of integers* is called an *integer-input problem*.

absolute value

For any instance x of an integer-input problem, we write $MaxInt(x)$ for the largest integer occurring in the input encoding. ◀

(As before, integers are encoded in binary.)

Integer Problems

Definition 3.1 (Integer-Input Problem)

A problem $L \subseteq U$ for which (part of the) input is a *sequence of integers* is called an *integer-input problem*.

For any instance x of an integer-input problem, we write $MaxInt(x)$ for the largest integer occurring in the input encoding. ◀

(As before, integers are encoded in binary.)

Examples

- TravelingSalesman
- SubsetSum
- BinPacking
- ILP
- Knapsack

Clicker Question

How large can $MaxInt(x)$ be as a function of $n = |x|$?



A	$O(1)$	**D**	$O(n^c)$	**G**	$2^{O(n)}$
B	$O(\log n)$	**E**	$O(n^{\log n})$	**H**	$O(2^{2^n})$
C	$O(n)$	**F**	$O(2^n)$	**I**	unbounded



→ *sli.do/advalg*

Clicker Question

How large can $MaxInt(x)$ be as a function of $n = |x|$?



- **A** $O(1)$ ✓
- **B** $O(\log n)$ ✓
- **C** $O(n)$ ✓
- **D** $O(n^c)$ ✓
- **E** $O(n^{\log n})$ ✓
- **F** $O(2^n)$ ✓
- **G** ~~$2^{O(n)}$~~ ✓
- **H** ~~$O(2^{2^n})$~~
- **I** ~~unbounded~~



→ *sli.do/advalg*

Pseudopolynomial

Definition 3.2 (Pseudopolynomial algorithm)

Let $L \subseteq U$ be an integer-input problem and A an algorithm that solves L.
A has *pseudopolynomial time for* U, if there is a polynomial p in two variables with

$$time_A(x) = O\Big(p\big(|x|, MaxInt(x)\big)\Big),$$

for every instance $x \in U$.

Pseudopolynomial

Definition 3.2 (Pseudopolynomial algorithm)

$\neq$ quasi-polynomial ($= 2^{O(\log^c n)}$)

Let $L \subseteq U$ be an integer-input problem and A an algorithm that solves L.
A has *pseudopolynomial time for* U, if there is a polynomial p in two variables with

$$time_A(x) \quad = \quad O\Big(p\big(|x|, MaxInt(x)\big)\Big),$$

for every instance $x \in U$. ◀

Note: If $MaxInt(x) \leq h(|x|)$ for a polynomial h, then
$p\big(|x|, MaxInt(x)\big) \leq g(|x|)$ for a polynomial g.

Pseudopolynomial Languages

Definition 3.3 (Number-bounded restriction)

Let $L \subseteq U$ be an integer-input problem and let $h : \mathbb{N} \to \mathbb{N}$ be weakly increasing. The *h-restriction of U*, written $U_{MaxInt \leq h}$, is U restricted to inputs x with $MaxInt(x) \leq h(|x|)$. ◀

Pseudopolynomial Languages

Definition 3.3 (Number-bounded restriction)

Let $L \subseteq U$ be an integer-input problem and let $h : \mathbb{N} \to \mathbb{N}$ be weakly increasing.
The *h-restriction of U*, written $U_{MaxInt \leq h}$, is U restricted to inputs x with $MaxInt(x) \leq h(|x|)$.

Theorem 3.4 (Pseudopolynomial is polynomial for small h)

Let $L \subseteq U$ be an integer-input problem and A a pseudopolynomial algorithm for L.
Then for every polynomial h there is a polytime algorithm for $U_{MaxInt \leq h}$.

Proof:

A has running time $O(p(|x|, MaxInt(x)))$ $\quad n = |x|$

$x \in U_{MaxInt \leq h} \Rightarrow MaxInt(x) \leq h(|x|) = O(n^c)$ for constant c

$\Rightarrow O(p(n, n^c)) = O(n^d)$ for constant d

Pseudopolynomial Languages

Definition 3.3 (Number-bounded restriction)
Let $L \subseteq U$ be an integer-input problem and let $h : \mathbb{N} \to \mathbb{N}$ be weakly increasing.
The *h-restriction of U*, written $U_{MaxInt \leq h}$, is U restricted to inputs x with $MaxInt(x) \leq h(|x|)$.

Theorem 3.4 (Pseudopolynomial is polynomial for small h)
Let $L \subseteq U$ be an integer-input problem and A a pseudopolynomial algorithm for L.
Then for every polynomial h there is a polytime algorithm for $U_{MaxInt \leq h}$.

Proof:

Hence if $L \subseteq U$ is a decision problem then $U_{MaxInt \leq h} \in \mathsf{P}$,
if it is an optimization problem then $U_{MaxInt \leq h} \in \mathsf{PO}$.

3.2 Knapsack

Knapsack (Optimization Version)

Definition 3.5 (Knapsack (Optimization Version))

Given: tuple $(w_1, \ldots, w_n; v_1, \ldots, v_n; b)$ of $2n + 1$ positive integers, $n \in \mathbb{N}$.
We call b the *capacity* of the knapsack, w_i the *weight* and v_i the *value* (profit) of the i-th object, $1 \leq i \leq n$.

Goal: The *optimization problem* KNAPSACK asks to find a subset $T \subseteq \{1, 2, \ldots, n\}$ of items with maximal total value $cost(T) = \sum_{i \in T} v_i$ such that T fits into the knapsack, i. e., $\sum_{i \in T} w_i \leq b$.

Recap: The 6 Steps of Dynamic Programming

⇝ see *Efficient Algorithms*

1. Define **subproblems** (and relate to original problem)
2. **Guess** (part of solution) ⇝ local brute force
3. Set up **DP recurrence** (for quality of solution)
4. Recursive implementation with **Memoization**
5. Bottom-up **table filling** (topological sort of subproblem dependency graph)
6. **Backtracing** to reconstruct optimal solution

- Steps 1–3 require insight / creativity / intuition;
  Steps 4–6 are mostly automatic / same each time

⇝ Correctness proof usually at level of DP recurrence


running time too! worst case time = #subproblems · time to find single best guess

Dynamic Programming Solution

- **Subproblems:** (n', b'): only items $1 \leq i \leq n'$ and total weight b'
- **Guess:** whether to include item n'
- **Recurrence:** $V[n', b'] =$ max value in subproblem (n', b')

subproblems = $n \times b$ — bounded only by Matlut

n: bounded by input size